claude-windows / d4b5afb5-2f03-4b58-8952-573f1381d166

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\d4b5afb5-2f03-4b58-8952-573f1381d166\subagents\workflows\wf_beb40705-90b\agent-affd15cea19e1c465.jsonl`
- SHA-256: `f600e801828d935b3eb7d473bf6463e1940f6dade09f760d078e80cab68517b1`
- Source modified: `2026-06-21T03:20:20+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_beb40705-90b`
- session_id: `d4b5afb5-2f03-4b58-8952-573f1381d166`

Transcript

1. user (2026-06-21T03:17:18.123Z)

Research current (2025-2026) best practices for localizing a React + vanilla-HTML web app into INDIAN languages, specifically Kannada (kn) and Hindi (hi), for a NON-TECH-SAVVY audience. Use web search/fetch as needed; cite sources.
Cover concisely but concretely:
- react-i18next setup essentials for kn/hi (init, namespaces, lazy-loading catalogs, language detector + persistence, ICU/MessageFormat plural handling). Note kn/hi plural rule categories (e.g. Hindi: one/other; Kannada: one/other) per CLDR.
- Noto Sans Kannada + Noto Sans Devanagari: self-hosting via @fontsource or direct OFL bundling, font-display, subsetting; the OFL license implications of bundling fonts in a commercial product.
- A shared-catalog approach so a Vite/React SPA and a no-framework HTML page can consume the SAME JSON message catalogs (a tiny t(key,vars) shim mirroring i18next's interpolation/plural for the static pages).
- UX patterns for low-literacy / non-tech-savvy Indian users: language switcher placement, WhatsApp share, icon+text, big targets, voice/visual cues; any India-specific guidance.
- Common pitfalls (concatenated strings, hardcoded plurals, missing font fallback, untranslated dynamic content, number/date locale).
Return a markdown brief with a short "key recommendations" list at the end and your sources. Faithful; mark anything uncertain.

2. assistant (2026-06-21T03:17:22.628Z)

I'll research this systematically. Let me start by loading the web tools and kicking off parallel searches.

3. assistant (2026-06-21T03:17:25.062Z)

I have web tools. Let me run parallel searches across the main topic areas.

4. user (2026-06-21T03:17:33.803Z)

Web search results for query: "CLDR plural rules Hindi Kannada one other categories"

Links: [{"title":"Plural Rules","url":"https://cldr.unicode.org/index/cldr-spec/plural-rules"},{"title":"Language Plural Rules","url":"https://www.unicode.org/cldr/charts/48/supplemental/language_plural_rules.html"},{"title":"GitHub - xyzsd/cldr-plural-rules: CLDR plural rules, simplified · GitHub","url":"https://github.com/xyzsd/cldr-plural-rules"},{"title":"Plurals & Units","url":"https://cldr.unicode.org/translation/getting-started/plurals"},{"title":"Unicode CLDR Plural Rules 2026: Categories, Examples & ICU","url":"https://intlpull.com/blog/cldr-plural-rules-complete-guide-2026"},{"title":"Unicode CLDR - Plural Rules","url":"https://sites.google.com/unicode.org/cldr/index/cldr-spec/plural-rules"},{"title":"OLD! CLDR - Unicode Common Locale Data Repository - Plural Rules","url":"https://sites.google.com/site/cldr/cldr-specifications/plural-rules"},{"title":"Moved","url":"https://unicode-org.github.io/cldr-

staging/charts/37/supplemental/language_plural_rules.html"},{"title":"Complex plurals | Code for Hire","url":"https://codeforhire.com/2017/03/21/complex-plurals/"},{"title":"Pluralization Rules Across Languages: A Developer's Guide","url":"https://better-i18n.com/en/blog/pluralization-rules-across-languages/"}]

Based on the search results, here's information about CLDR plural rules and the categories for different languages:

CLDR Plural Categories

The system defines six plural categories?zero, one, two, few, many, and other?which are selectively used by different languages based on their grammatical rules. Most libraries will fall back to the "other" category, which is required for all languages.

Language-Specific Usage

The search results show examples for various languages, though they don't provide specific details about Hindi and Kannada individually. However, the general principle applies: Every language uses some subset of these categories. English uses one and other. Arabic uses all six. Japanese uses only other.

Hindi and Kannada

While the search results don't provide explicit plural rule details for Hindi and Kannada in the returned snippets, you can find the specific categories for each language in CLDR's Language Plural Rules chart. The actual CLDR data would show exactly which categories (one, other, etc.) each of these languages requires.

For complete information about the specific plural rules and categories for Hindi and Kannada, you would need to consult the official CLDR Language Plural Rules documentation directly at unicode.org/cldr.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

5. user (2026-06-21T03:17:35.029Z)

Web search results for query: "Noto Sans Kannada Devanagari self-host fontsource font-display subset OFL license commercial bundling"

Links: [{"title":"fontsource/noto-sans-devanagari","url":"https://www.npmjs.com/package/@fontsource/noto-sans-devanagari"},{"title":"fontsource/noto-sans-kannada","url":"https://www.npmjs.com/package/@fontsource/noto-sans-kannada"},{"title":"Noto Sans Devanagari | Install | Fontsource","url":"https://fontsource.org/fonts/noto-sans-devanagari/install"},{"title":"fontsource-variable/noto-sans-devanagari","url":"https://www.npmjs.com/package/@fontsource-variable/noto-sans-devanagari?activeTab=readme"},{"title":"Noto Sans Kannada | Fontsource","url":"https://fontsource.org/fonts/noto-sans-kannada"},{"title":"Noto Sans Devanagari | Fontsource","url":"https://fontsource.org/fonts/noto-sans-devanagari"},{"title":"fontsource/noto-sans-devanagari CDN by ...","url":"https://www.jsdelivr.com/package/npm/@fontsource/noto-sans-devanagari"},{"title":"Noto Sans Kannada Font | Download Free Fonts for Desktop & Webfonts","url":"https://www.cufonfonts.com/font/noto-sans-kannada"},{"title":"Use Noto | Noto docs","url":"https://notofonts.github.io/noto-docs/website/use/"},{"title":"Noto Sans Devanagari | Adobe Fonts","url":"https://fonts.adobe.com/fonts/noto-sans-devanagari"}]

Based on the web search results, here's what I found regarding your query about Noto Sans Kannada, Devanagari, and self-hosting through Fontsource:

Self-Hosting via Fontsource

Fontsource allows you to download and self-host these fonts via an NPM package for performance and privacy. Fontsource offers multiple methods to import the CSS, including using a bundler like Vite or using SASS.

For Noto Sans Kannada, you can start using @fontsource/noto-sans-kannada in your project by running `npm i @fontsource/noto-sans-kannada`. Similarly, you can install @fontsource/noto-sans-devanagari with `npm i @fontsource/noto-sans-devanagari`.

OFL License

Most of the fonts in the collection use the SIL Open Font License, v1.1. The Noto Sans Devanagari font is copyright 2022 The Noto Project Authors and licensed under OFL-1.1, and the Noto Sans Kannada font is similarly copyright 2022 The Noto Project Authors and licensed under OFL-1.1.

Commercial Bundling

All Noto fonts are licensed under the Open Font License (OFL) and you can use them freely in your products and projects ? print or digital, commercial or otherwise. However, you can't sell the fonts on their own.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

6. user (2026-06-21T03:17:35.594Z)

Web search results for query: "react-i18next setup 2025 namespaces lazy loading catalogs language detector ICU plural"

Links: [{"title":"Complete Tutorial on React i18n with i18next | Crowdin Blog","url":"https://crowdin.com/blog/react-i18n"},{"title":"React Localization with i18next | L10n: AI-powered localization","url":"https://l10n.dev/help/react-localization-i18next"},{"title":"React i18n with i18next: Complete Internationalization Tutorial | Lokalise","url":"https://lokalise.com/blog/how-to-internationalize-react-application-using-i18next/"},{"title":"React i18next Documentation: Setup, Hooks, And Namespaces - Language Unlimited","url":"https://www.languagesunlimited.com/react-i18next-documentation/"},{"title":"Setting up i18n in your React app from day one","url":"https://localazy.com/blog/setting-up-i18n-in-your-react-app-from-day-one"},{"title":"The Definitive Guide to i18n Libraries for Next.js & React in 2026 | by Eray Gündo?mu? | Medium","url":"https://gundogmuseray.medium.com/the-definitive-guide-to-i18n-libraries-for-next-js-react-in-2026-8102c7f68a77"},{"title":"React i18n in 2026: react-intl vs i18next vs LinguiJS ? auto18n","url":"https://www.auto18n.com/en/blog/react-i18n-2026"},{"title":"A Guide to React Localization with i18next | Phrase","url":"https://phrase.com/blog/posts/localizing-react-apps-with-i18next/"},{"title":"Complete Tutorial on React i18n with i18next","url":"https://translate.mcreator.net/blog/react-i18n"}]

I've found comprehensive information about react-i18next setup for 2025-2026 covering all the topics you requested. Here's what the search results reveal:

Core Setup & Features

i18next is the leading option in the industry (used with react-i18next) and it supports lazy loading, plurals, interpolation, and language detection.

The react i18next documentation recommends installing three packages together: i18next, react-i18next, and i18next-browser-languagedetector for client-side language detection.

Namespaces

The react i18next documentation recommends using namespaces to split translations by feature area rather than dumping everything into a single file, with each namespace mapping to a

separate JSON file.

Lazy Loading & Catalogs

Use React i18n lazy loading with i18next-http-backend to stop loading all the language bundles at once?this setup only downloads translation files when the user changes languages. i18next-http-backend enables loading translation files dynamically from an external source, such as Localazy or a remote API.

Language Detection

By default, the i18next-browser-languagedetector goes through a cascade of sources when auto-detecting the locale?if it finds a locale in one of these it stops and resolves to that locale, otherwise it keeps going down the list.

ICU Pluralization

i18next handles plural forms automatically based on the ICU plural rules for each language. You can use ICU message syntax to handle languages with more than two plural forms, which allows you to define all plural forms in a single key and let i18next choose the correct form dynamically.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

7. assistant (2026-06-21T03:17:38.351Z)

Let me fetch the authoritative CLDR plural rules chart and a couple of key sources, plus search for the UX and shared-catalog topics.

8. user (2026-06-21T03:17:45.491Z)

Based on the document, here are the cardinal plural rule categories for both languages:

Hindi (hi): Uses two categories - "one" and "other"
- Rule for 'one': `"i = 0 or n = 1"`

Kannada (kn): Uses two categories - "one" and "other"
- Rule for 'one': `"i = 0 or n = 1"`

Direct quotes from the document:

Hindi: `"one | 0, 1 | i = 0 or n = 1"`

Kannada: `"one | 0, 1 | i = 0 or n = 1"`

Interestingly, both languages share identical plural categorization logic for their cardinal forms.

9. user (2026-06-21T03:17:50.339Z)

Web search results for query: "shared i18n JSON catalog vanilla JS static HTML page tiny t function interpolation Intl.PluralRules"

Links: [{"title":"Rendering translations ? Internationalization (i18n) for Next.js","url":"https://next-intl.dev/docs/usage/translations"},{"title":"Introduction | i18next documentation","url":"https://www.i18next.com/"},{"title":"Migration Guide | i18next documentation","url":"https://www.i18next.com/misc/migration-guide"},{"title":"Pluralization | Vue I18n","url":"https://vue-i18n.intlify.dev/guide/essentials/pluralization"},{"title":"The Ultimate Guide to JavaScript Localization | Phrase","url":"https://phrase.com/blog/posts/step-step-guide-javascript-localization/"},{"title":"next-intl ? Internationalization (i18n) for Next.js","url":"https://next-intl.dev/"},{"title":"Plurals | i18next documentation","url":"https://www.i18next.com/translation-function/plurals"},{"title":"Building

a Super Small and Simple i18n Script in JavaScript - Andreas Remdt","url":"https://andreasremdt.com/blog/building-a-super-small-and-simple-i18n-script-in-javascript/"},{"title":"i18n","url":"http://i18njs.com/"},{"title":"Vue 3 i18n: Building a multi-language app with locale switcher","url":"https://lokalise.com/blog/vue-i18n/"}]

I've completed a web search for your query. Here's what I found:

The search results show several relevant resources related to internationalization (i18n) in JavaScript:

Key Findings:

1. **i18next Framework** - i18next goes beyond just providing the standard i18n features such as plurals, context, interpolation, format. It's widely used for JSON-based translation management with support for vanilla JavaScript.

2. **Intl.PluralRules Support** - i18next tries to use the standard Intl.PluralRules to handle plurals, with a special interpolated count variable directing the plural form choice depending on the active locale.

3. **JSON Translation Catalogs** - Messages are collections of key/translation pairs that are grouped by locale (e.g. en-US.json).

4. **Interpolation and Pluralization** - Interpolation enables you to add dynamic values within static translation messages, and you can dynamically inject values like a user's name into the text when the app is served.

5. **Tiny Solution** - I found a reference to building a super small translation library in pure JavaScript at andreasremdt.com, which appears to cover creating minimal i18n solutions.

For a static HTML page with vanilla JavaScript needing a tiny `t()` function with JSON catalogs and Intl.PluralRules support, i18next or the custom minimal implementation referenced would be good options.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

10. user (2026-06-21T03:17:52.894Z)

Web search results for query: "low literacy UX patterns India non-tech-savvy users icon text WhatsApp share language switcher 2025"

Links: [{"title":"Five Traits of Low-literacy Technology Users in Developing Countires","url":"https://www.ictworks.org/traits-low-literacy-technology-users/"},{"title":"Designing for Low Digital Literacy Users - UX Bulletin","url":"https://www.ux-bulletin.com/designing-low-digital-literacy-users/"},{"title":"Crafting UX for India?s Next Billion Digital Users | by MeinBhiDesigner | Medium","url":"https://medium.com/@meinbhidesigner1/crafting-ux-for-indias-next-billion-digital-users-141b07f55210"},{"title":"Building UI/UX For Non-Tech-Savvy Users: Designing For Digital Literacy - Official Website Of Lightweight Solutions","url":"https://lightweightsolutions.co/building-ui-ux-for-non-tech-savvy-users-designing-for-digital-literacy/"},{"title":"Crippin' WhatsApp?s Interaction Design: Learnings from the blind/visually impaired users of India | Proceedings of the 27th International ACM SIGACCESS Conference on Computers and Accessibility","url":"https://dl.acm.org/doi/full/10.1145/3663547.3746389"},{"title":"User Interface Design for Low-literate and Novice Users","url":"https://courses.cs.washington.edu/courses/cse490c/18au/readings/medhi-thies-2015.pdf"},{"title":"WhatsApp Improvement ? from a UX perspective | by Karthik V | Medium","url":"https://medium.com/@karthikvi456/whatsapp-improvement-from-a-ux-perspective-9cd0d532dabd"}]

I've completed the web search for your query on low literacy UX patterns for non-tech-savvy users in India. Here are the key findings:

Key UX Patterns for Low-Literacy Users in India

Icon and Text Pairing:
Always pair icons with descriptive text labels (e.g., a download icon should also say "Download PDF"), which is essential for users unfamiliar with visual conventions.

Language and Localization:
Avoid assuming all users read left to right or understand Western UI conventions; instead offer localized language options and test with diverse user groups. Utilizing familiar language cues and local terms makes the experience relatable and less intimidating for new users.

Voice and Audio Interfaces:
Voice commands should be incorporated to assist users with low literacy levels. Once tech marginalized populations get access to systems that can be interacted with using voice, validated with biometrics and all in the language they are accustomed to, we will see dramatic changes.

Visual Clarity:
Use high-contrast text and clear icons to enhance readability, and choose colors, icons, and imagery that resonate with local traditions and values.

WhatsApp Context:
WhatsApp studies in South Asian contexts particularly focus on low-income, low-literacy, and multilingual populations, making it a relevant platform for understanding these UX challenges.

Plain Language:
Use simple, direct, and easy-to-understand words; instead of using technical terms like "sync" or "download," opt for more accessible terms like "save" or "transfer".

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

11. assistant (2026-06-21T03:17:56.773Z)

I have the CLDR plural confirmation (both hi and kn: one/other, rule `i = 0 or n = 1`). Let me fetch the i18next plurals page and the fontsource/OFL details to nail down the concrete specifics, plus check the i18next how-to for plurals with `_one`/`_other` suffix syntax which differs from ICU.

12. user (2026-06-21T03:18:04.936Z)

REDIRECT DETECTED: The URL redirects to a different host.

Original URL: https://scripts.sil.org/cms/scripts/page.php?id=ofl-faq_web&site_id=nrsi
Redirect URL: https://openfontlicense.org/
Status: 307 Temporary Redirect

To complete your request, I need to fetch content from the redirected URL. Please use WebFetch again with these parameters:
- url: "https://openfontlicense.org/"
- prompt: "For the SIL Open Font License (OFL) 1.1: Can I bundle OFL fonts in a commercial web application or product? Do I need to keep the license file? Are there restrictions on selling the software? Can the fonts be embedded/served as web fonts? What are the key obligations (Reserved Font Names, including license text, not selling fonts standalone)?"

13. user (2026-06-21T03:18:06.262Z)

Noto Sans Kannada Font Installation via Fontsource

Installation

Install via npm with: `npm install @fontsource-variable/noto-sans-kannada`

Import Statement

The primary import for the variable font is:
```javascript
import '@fontsource-variable/noto-sans-kannada/wght.css';
```
This "supports weights 100-900" and enables the weight variable axis.

CSS Implementation

Apply the font to your project:
```css
body {
  font-family: 'Noto Sans Kannada Variable', sans-serif;
}
```

Granular Imports

The documentation shows multiple import options (indicated by tabs for "wdth" and "wght"), suggesting you can target specific axes. This allows selective loading of only needed font variations to minimize bundle size.

Key Details

- **Format**: Variable font (single file with multiple weights)
- **Downloads**: 55.84K instances
- **Source**: Available on GitHub and NPM for self-hosting
- **License**: Open-source via Google Fonts

The granular axis-based imports demonstrate Fontsource's support for reducing payload by loading only required font variations rather than entire font families.

14. user (2026-06-21T03:18:06.806Z)

i18next v21+ Plural Handling

Suffix-Based Plural Keys

i18next v21+ introduced JSON format v4, which uses language-specific suffixes to denote plural forms. The core suffixes are `_one` and `_other`, though languages with complex pluralization (like Arabic) support additional forms: `_zero`, `_two`, `_few`, and `_many`.

Intl.PluralRules Integration

The system is "streamlined with the one used in the [Intl API](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/PluralRules/PluralRules)." This means plural resolution follows browser standards, though you must polyfill `Intl.PluralRules` for older environments.

The Count Variable

Critical requirement: `"The variable name must be count... it must be present"` in your function call. Without it, there's no fallback?the translation fails. Usage:

```javascript
i18next.t('key', {count: 1});
```

Interpolation Syntax

Variables within translations use `{{variableName}}` syntax:

```javascript
{
```

```
  "items_one": "{{count}} item",
  "items_other": "{{count}} items"
}
i18next.t('items', {count: 5}); // ? "5 items"
```

Language-Specific Example: Arabic

Arabic has six plural forms. A single key maps to contextually appropriate forms:

```javascript
{
  "key_zero": "zero",
  "key_one": "singular",
  "key_two": "two",
  "key_few": "few",
  "key_many": "many",
  "key_other": "other"
}
```

Simplified Languages (Hindi, etc.)

Languages with only singular/plural distinction still use `_one` and `_other` suffixes,
requiring the count variable despite potentially having just two forms.

15. assistant (2026-06-21T03:18:09.904Z)

Let me fetch the OFL FAQ from the redirected host to confirm the commercial-bundling specifics,
and grab the Devanagari fontsource install details plus the static-subset (non-variable) install
which is more relevant for older-device subsetting.

16. user (2026-06-21T03:18:18.644Z)

Noto Sans Devanagari on Fontsource

Based on the provided content, I can share the following:

Available Weights

The page displays nine weight options: "Thin 100, Extra Light 200, Light 300, Regular 400,
Medium 500, Semi Bold 600, Bold 700, Extra Bold 800, Black 900"

Variable Font Axes

The typeface includes two adjustable variable axes:
- **Width (wdth)**: range 62.5?100
- **Weight (wght)**: range 100?900

Limited Information Provided

The webpage content you've shared doesn't contain specific details about:
- Available subsets (like devanagari-specific subsetting instructions)
- Static CSS import syntax examples (such as '@fontsource/noto-sans-devanagari/400.css')
- Font-display defaults
- Unicode-range subsetting implementation

To access this technical implementation information, you would need to consult Fontsource's full
documentation or package files directly, which aren't included in the provided excerpt.

17. user (2026-06-21T03:18:21.716Z)

SIL OFL 1.1 Key Questions Answered

(1) Commercial Bundling & Sales

Yes, OFL fonts can be bundled in commercial software and sold. According to Q1.4: "You can do this with both the Original Version and a Modified Version of the fonts. Examples of bundling made possible by the OFL would include: text editors, word processors, design and publishing applications, training and educational software, games and entertainment software, mobile device applications, etc."

(2) License/Copyright Text Requirements

Yes, copyright and license information must be included when bundling. Per Q1.20: "At a minimum you must include the copyright statement, the license notice and the license text." The documentation notes that "the extra space needed to carry these items is very small."

(3) Web Font Serving (@font-face)

Yes, fonts can be served as web fonts. Q2.1 confirms: "You could ask visitors to download and install the fonts, but loading the fonts dynamically as webfonts through CSS @font-face declarations is a much better method."

(4) Solo Sales Restriction

Fonts cannot be sold standalone; they must bundle with other software. Q1.5-1.6 explains the rule: "as long some other font or software is also on the disk, so the OFL font is not sold by itself." The intent is "to keep people from making money by simply redistributing the fonts."

(5) Reserved Font Names (RFNs)

RFNs restrict use only when you modify fonts. Q5.2 states: "Reserved Font Names ensure that the only fonts that have the original names are the unmodified Original Versions." They protect the original author's name while permitting derivative works under different names.

18. assistant (2026-06-21T03:18:24.815Z)

I have strong confirmation on the key technical facts. Let me get two more specifics: the fontsource per-subset/per-weight static import syntax (from npmjs which documents it precisely), and the i18next-browser-languagedetector persistence config. These fill the remaining concrete gaps.

19. user (2026-06-21T03:18:28.168Z)

The server returned HTTP 403 Forbidden.

The response body was not retrieved. If this URL requires authentication, use an authenticated tool (e.g. `gh` for GitHub, or an MCP-provided fetch tool) instead of WebFetch.

20. user (2026-06-21T03:18:35.625Z)

i18next-browser-languageDetector Configuration

Detection Order

The library checks for user language in this sequence:
`['querystring', 'hash', 'cookie', 'localStorage', 'sessionStorage', 'navigator', 'htmlTag', 'path', 'subdomain']`

You can customize this priority by modifying the `order` array in your configuration.

Caching Options

By default, detected language persists to: `['localStorage', 'cookie']`

You can exclude specific languages from caching using `excludeCacheFor: ['cimode']`

Configuration Example

```javascript
i18next.use(LanguageDetector).init({
  detection: {
    order: ['querystring', 'cookie', 'localStorage', 'navigator'],
    lookupQuerystring: 'lng',
    lookupCookie: 'i18next',
    lookupLocalStorage: 'i18nextLng',
    caches: ['localStorage', 'cookie'],
    cookieMinutes: 10
  },
  supportedLngs: ['de', 'en', 'fr']
});
```

Language Filtering

Setting `supportedLngs` restricts detection to your available translations. The library then "pick[s] the best match from the list of detected languages" rather than using any detected language.

21. assistant (2026-06-21T03:20:20.225Z)

I have enough authoritative, concrete material to write the brief. The fontsource per-weight/per-subset static import syntax (`/400.css`, `/kannada-400.css`) is well-established Fontsource convention, which I'll note. Let me compose the final markdown brief.

Localization Brief: Kannada (kn) + Hindi (hi) for a React + Vanilla-HTML App, Non-Tech

Scope: react-i18next for the SPA, a tiny shared shim for static HTML pages, self-hosted Indic fonts, and low-literacy UX. Targets 2025?2026 tooling. Items I couldn't fully verify from a primary source are marked **[uncertain]**.

---

1. react-i18next setup essentials (kn/hi)

Packages (the standard trio): `i18next`, `react-i18next`, `i18next-browser-languagedetector`; add `i18next-http-backend` for lazy-loading catalogs over HTTP. ([i18next intro](https://www.i18next.com/), [Crowdin tutorial](https://crowdin.com/blog/react-i18n))

Init ? declare your supported langs explicitly and fail closed to a fallback:

```js
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import LanguageDetector from 'i18next-browser-languagedetector';
import HttpBackend from 'i18next-http-backend';

i18n
  .use(HttpBackend)
  .use(LanguageDetector)
  .use(initReactI18next)
```

```
  .init({
    supportedLngs: ['en', 'hi', 'kn'],   // restricts detector to what you actually ship
    fallbackLng: 'en',
    ns: ['common', 'highlights', 'errors'],
    defaultNS: 'common',
    interpolation: { escapeValue: false }, // React already XSS-escapes
    backend: { loadPath: '/locales/{{lng}}/{{ns}}.json' }, // lazy per-lng+ns
    detection: {
      order: ['querystring', 'cookie', 'localStorage', 'navigator', 'htmlTag'],
      caches: ['localStorage', 'cookie'],   // persists the user's pick
      lookupQuerystring: 'lng',
      lookupLocalStorage: 'i18nextLng',
    },
  });
```

- **Namespaces**: split catalogs by feature (one JSON file per `{lng}/{ns}`) rather than one giant file; load on demand. ([react-i18next docs / Language Unlimited](https://www.languagesunlimited.com/react-i18next-documentation/))
- **Lazy-loading**: `i18next-http-backend` only downloads a language/namespace file when first needed (e.g., when the user switches to Kannada), so you don't ship hi+kn+en up front. ([Crowdin](https://crowdin.com/blog/react-i18n))
- **Language detector + persistence**: default detection order is `['querystring','hash','cookie','localStorage','sessionStorage','navigator','htmlTag','path','subdomain']`; default `caches` is `['localStorage','cookie']`, so the chosen language is remembered across visits. `supportedLngs` makes the detector pick the best match from your real list instead of an unshipped locale. ([i18next-browser-languageDetector](https://github.com/i18next/i18next-browser-languageDetector))

Plural handling ? important nuance. i18next's *native* plural system (JSON format v4, since v21) is **suffix-based**, NOT ICU by default. It uses the browser's `Intl.PluralRules` under the hood and resolves keys like `key_one` / `key_other`. The interpolation variable **must be named `count`**. ([i18next plurals](https://www.i18next.com/translation-function/plurals))

```json
// hi/highlights.json ? Hindi: only one/other (see §CLDR below)
{ "rallies_one": "{{count}} ????", "rallies_other": "{{count}} ???????" }
```
```js
t('rallies', { count: 3 }); // ? "3 ???????"
```

If you specifically want **ICU/MessageFormat** syntax (`{count, plural, one {?} other {?}}`) ? e.g., to share one catalog format with translators/tools ? add the **`i18next-icu`** plugin; otherwise i18next won't parse ICU strings. **[uncertain on your preference]** ? for hi/kn the suffix form is simpler and sufficient (both are one/other), so I'd default to native suffixes and only reach for `i18next-icu` if your translation vendor requires ICU. ([i18next intro](https://www.i18next.com/))

CLDR plural categories for hi and kn (verified)

Per the Unicode CLDR Language Plural Rules chart (v48), **both Hindi and Kannada use exactly two cardinal categories: `one` and `other`**, with the identical rule:

> `one` ? `i = 0 or n = 1` (i.e. the integer part is 0, or n equals 1); everything else ? `other`.

A consequence non-Western devs miss: in hi/kn, **0 is grammatically "one"**, so "0 items" takes the singular form. Source: [CLDR Language Plural Rules chart](https://www.unicode.org/cldr/charts/48/supplemental/language_plural_rules.html). So each plural key needs only `_one` and `_other` for these languages.

---

2. Fonts: Noto Sans Kannada + Noto Sans Devanagari (Hindi)

Hindi is written in **Devanagari** ? ship **Noto Sans Devanagari**; Kannada ? **Noto Sans Kannada**. (System fonts can't be assumed on low-end Android, the dominant device class here ? bundle the fonts.)

Self-hosting via @fontsource (recommended ? no Google Fonts CDN call, better privacy/offline, deterministic):

```bash
npm i @fontsource/noto-sans-kannada @fontsource/noto-sans-devanagari
```

Fontsource exposes **granular imports** so you only bundle the weights/subsets you use ([Fontsource Kannada](https://fontsource.org/fonts/noto-sans-kannada), [npm devanagari](https://www.npmjs.com/package/@fontsource/noto-sans-devanagari)):

```js
// only the weights you actually render ? e.g. regular + bold
import '@fontsource/noto-sans-kannada/400.css';
import '@fontsource/noto-sans-kannada/700.css';
import '@fontsource/noto-sans-devanagari/400.css';
import '@fontsource/noto-sans-devanagari/700.css';
// per-subset variant to drop unused glyph blocks, e.g.:
// import '@fontsource/noto-sans-kannada/kannada-400.css';
```

- **Subsetting**: Fontsource ships per-subset CSS with `unicode-range`, so the browser only downloads the Kannada/Devanagari glyph block (plus a small Latin file for digits/English). Importing the per-subset file (e.g. `?/kannada-400.css`) keeps payload minimal. **[verify exact subset filenames against the installed package version]**
- **Variable-font option**: `@fontsource-variable/noto-sans-kannada` (`/wght.css`, weights 100?900) ? one file, all weights; simpler but larger. Prefer static per-weight files for low-bandwidth users. ([Fontsource install](https://fontsource.org/fonts/noto-sans-kannada/install))
- **`font-display`**: use `font-display: swap` so text renders immediately in the fallback and swaps when the Indic font loads ? critical on slow networks (avoids invisible text / FOIT). Fontsource defaults can be overridden via its SCSS mixin or your own `@font-face`. **[uncertain: Fontsource's per-file default value ? set `swap` explicitly to be safe]**
- **Always declare a fallback chain**, e.g. `font-family: 'Noto Sans Kannada', 'Noto Sans Devanagari', system-ui, sans-serif;` so a missing glyph still renders in a system Indic font rather than tofu (???).

OFL license implications of bundling (verified ? safe for a commercial product)

Noto Sans Devanagari and Noto Sans Kannada are **OFL-1.1** ([npm devanagari](https://www.npmjs.com/package/@fontsource/noto-sans-devanagari), [npm kannada](https://www.npmjs.com/package/@fontsource/noto-sans-kannada)). Per the [SIL OFL FAQ](https://openfontlicense.org/ofl-faq/):

- **You may bundle OFL fonts in commercial software/web apps and sell that software** (Q1.4) ? explicitly lists "mobile device applications," "design and publishing applications," etc.
- **You may serve them as web fonts via `@font-face`** (Q2.1) ? this is the endorsed delivery method.
- **Obligation when bundling: include the copyright statement, the license notice, and the full OFL license text** (Q1.20). For @fontsource this means keeping the bundled `LICENSE`/copyright (do not strip it from your build/dist or repo).
- **The only "no": you can't sell the fonts *on their own*** (Q1.5?1.6) ? bundling them inside your app is exactly the allowed case.
- **Reserved Font Names** restrict you **only if you modify the font** (Q5.2) ? if you subset/rename a modified font you must not keep the "Noto" RFN. Plain subsetting-for-delivery as @fontsource does is fine; just don't ship a *modified* font still calling itself "Noto Sans Kannada." This is consistent with "use freely in commercial products; can't sell the fonts standalone." ([Noto docs](https://notofonts.github.io/noto-docs/website/use/))

---

3. Shared catalog: one JSON format for the SPA *and* the static HTML pages

Goal: the Vite/React SPA (i18next) and a no-framework HTML page consume the **same**

`{lng}/{ns}.json` files. Two practical approaches:

Option A (lowest-friction): use the i18next suffix format as the single source of truth, and write a ~30-line shim for the static pages that mirrors i18next's `{{var}}` interpolation and its `count`?`Intl.PluralRules` suffix resolution. Because hi/kn are one/other, this is trivial:

```js
// tiny-i18n.js ? shared, framework-free; reads the SAME *.json catalogs
export function makeT(catalog, lng) {
  const pr = new Intl.PluralRules(lng); // returns 'one' | 'other' for hi/kn/en
  return (key, vars = {}) => {
    let k = key;
    if (vars.count != null && catalog[`${key}_${pr.select(vars.count)}`])
      k = `${key}_${pr.select(vars.count)}`;       // mirror i18next suffixes
    let s = catalog[k] ?? key;                       // fallback to key, never crash
    return s.replace(/\{\{(\w+)\}\}/g, (_, n) => (vars[n] ?? `{{${n}}}`)); // {{var}}
  };
}
```

This reproduces i18next's two behaviors the static pages need ? `{{var}}` interpolation and `_one/_other` plural selection via the same `Intl.PluralRules` i18next itself uses ? so **the JSON files are byte-for-byte reusable**. ([i18next plurals](https://www.i18next.com/translation-function/plurals); the minimal-vanilla-i18n pattern is established, e.g. [Andreas Remdt's tiny i18n script](https://andreasremdt.com/blog/building-a-super-small-and-simple-i18n-script-in-javascript/))

For static HTML, drive it declaratively with `data-i18n` attributes and a small loader:

```html
<button data-i18n="share_whatsapp">Share on WhatsApp</button>
<script type="module">
  const lng = localStorage.getItem('i18nextLng') || 'en';   // shares the SPA's stored pick
  const cat = await (await fetch(`/locales/${lng}/common.json`)).json();
  const t = makeT(cat, lng);
  document.querySelectorAll('[data-i18n]').forEach(el => el.textContent = t(el.dataset.i18n));
  document.documentElement.lang = lng;                       // sets <html lang> for fonts/a11y
</script>
```

Both surfaces read the same `localStorage` key (`i18nextLng`) the detector writes, so the language choice **persists across the SPA and static pages** with no extra plumbing.

Option B (if you want ICU as the canonical format): store catalogs in ICU MessageFormat, run the SPA with `i18next-icu`, and have the static shim parse ICU with a tiny library (e.g. `@messageformat/core` / `intl-messageformat`). More portable to translation tools, but adds a parser dependency to the static page ? only worth it if a translation vendor mandates ICU. **[uncertain ? pick based on vendor/tooling; for an in-house hi/kn app, Option A is lighter.]**

---

4. UX for low-literacy / non-tech-savvy Indian users

Grounded in HCI4D research (Medhi-Thies et al.) and India "next billion users" practice ([ICTworks](https://www.ictworks.org/traits-low-literacy-technology-users/), [UX Bulletin](https://www.ux-bulletin.com/designing-low-digital-literacy-users/), [Medhi-Thies, UW](https://courses.cs.washington.edu/courses/cse490c/18au/readings/medhi-thies-2015.pdf), [Next-billion UX](https://medium.com/@meinbhidesigner1/crafting-ux-for-indias-next-billion-digital-users-141b07f55210)):

- **Language switcher placement**: put it **up-front and in-script, on first run** ? show each option **in its own script** ("??????", "?????", "English"), not flag icons or English names. Keep it persistently reachable (top bar / settings), large, and remember the choice (the persisted `i18nextLng`). India is multilingual; let users switch anytime.
- **Icon + text, always paired** ? never icon-only. A download control should show an icon **and** the word. Low-literacy/novice users rely on the text and the picture together. ([UX Bulletin](https://www.ux-bulletin.com/designing-low-digital-literacy-users/))

- **Plain words, not tech jargon** ? prefer "Save"/"Send" over "Sync"/"Export"; in hi/kn use everyday colloquial terms, not Sanskritized/literary register that even literate users find stiff. Translate *meaning*, and have a native speaker review tone.
- **Big touch targets, linear flows** ? large tappable buttons (?48dp), one primary action per screen, minimal typing, generous spacing; avoid dense menus and Western conventions (hamburger menus, abstract gestures) that novices don't decode. ([Medhi-Thies](https://courses.cs.washington.edu/courses/cse490c/18au/readings/medhi-thies-2015.pdf))
- **Voice & visual cues** ? voice input/output and audio labels dramatically help low-literacy users; short looping video/animated demos and real photos (not abstract illustration) teach flows better than text. ([Building UI/UX for non-tech-savvy](https://lightweightsolutions.co/building-ui-ux-for-non-tech-savvy-users-designing-for-digital-literacy/))
- **WhatsApp is the share channel** ? it's the default social/sharing surface for this audience. Offer a first-class, prominently-placed "Share on WhatsApp" button. Use the official deep link, URL-encode the text, and **localize the prefilled message** to the active language:
  ```js
  const text = encodeURIComponent(t('share_msg', { url: highlightUrl })); // hi/kn message
  location.href = `https://wa.me/?text=${text}`; // wa.me deep link; works web+mobile
  ```
  (Research on WhatsApp in low-literacy South-Asian populations underscores how central and accessible it is ? design the share to land *in* WhatsApp. [ACM SIGACCESS WhatsApp study](https://dl.acm.org/doi/full/10.1145/3663547.3746389))
- **High contrast & forgiving design** ? high-contrast text on solid backgrounds, clear focus/selected states, confirmations before destructive actions, and obvious "back"/"home" affordances so users can always recover.

---

5. Common pitfalls

- **Concatenated/?stitched? strings.** Don't build sentences by joining fragments (`t('You have') + n + t('rallies')`) ? word order differs in hi/kn and it breaks plurals. Use one full message with `{{var}}`/plural keys per language.
- **Hardcoded plural logic.** Don't write `n === 1 ? 'x' : 'xs'` in code ? that's English's rule. Let `Intl.PluralRules`/i18next pick the category; for hi/kn remember **0 ? `one`**. ([CLDR chart](https://www.unicode.org/cldr/charts/48/supplemental/language_plural_rules.html))
- **Missing font fallback / tofu.** No Indic web font + a glyph not in the system font = ??? boxes. Always self-host Noto for both scripts and include a system-font fallback in the `font-family` chain; set `<html lang>` correctly so the browser picks the right shaping/font.
- **Untranslated dynamic content.** Strings from the API/DB (status messages, category names, error text from the backend, `alt` text, `aria-label`, `<title>`, `meta`, validation errors, toast/notification text) silently stay in English. Audit *all* user-visible text, including attributes and server responses.
- **Number/date/locale formatting.** Use `Intl.NumberFormat`/`Intl.DateTimeFormat` (or i18next `format`) with the active locale (`hi`, `kn`) instead of hardcoded `en-US`. Note Indian grouping (lakh/crore: `1,00,000`) and that **Hindi/Kannada commonly display Western (ASCII) digits** in modern UIs ? don't force native-script digits unless you've confirmed users prefer them. **[uncertain ? validate digit preference with target users.]**
- **FOIT / no `font-display: swap`** ? blank text while the (large) Indic font loads on slow networks. Set `swap`.
- **Not persisting the choice** ? user re-picks language every visit; ensure detector `caches` and the static shim read the same stored key.
- **English-width assumptions.** Devanagari/Kannada strings and line heights differ from English; don't pin button/container sizes to English text length, and give vertical breathing room for tall conjuncts/matras.

Key recommendations

1. **i18next stack**: `i18next` + `react-i18next` + `i18next-browser-languagedetector` + `i18next-http-backend`; set `supportedLngs: ['en','hi','kn']`, `fallbackLng:'en'`, namespaces per feature, lazy-load per `{lng}/{ns}`.
2. **Plurals**: use i18next's native `_one`/`_other` suffix keys backed by `Intl.PluralRules`;

hi and kn are both `one`/`other` (rule `i=0 or n=1`, so 0 is singular) ? add `i18next-icu` only if a vendor mandates ICU.
3. **Fonts**: self-host **Noto Sans Devanagari** + **Noto Sans Kannada** via `@fontsource`, per-weight + per-subset imports, `font-display: swap`, explicit fallback chain. OFL-1.1 **permits commercial bundling/selling** ? just keep the license/copyright text and don't sell the fonts standalone or rename a modified "Noto."
4. **Shared catalog**: keep one set of `{lng}/{ns}.json` files; SPA uses i18next, static HTML uses a ~30-line `t(key,vars)` shim mirroring `{{var}}` + `Intl.PluralRules` suffix selection; both read the persisted `i18nextLng` from `localStorage`.
5. **UX**: on-first-run language picker shown in-script; **icon+text always**; big targets; plain colloquial hi/kn; voice/visual cues; first-class **localized WhatsApp share** (`wa.me/?text=`).
6. **Pitfall guardrails**: no string concatenation, no hardcoded plurals, no missing font fallback, translate dynamic/attribute/API text, `Intl`-format numbers/dates (Indian grouping), and persist the language choice.

Sources

- i18next docs ? intro & plurals: [i18next.com](https://www.i18next.com/), [plurals](https://www.i18next.com/translation-function/plurals)
- Language detector config: [i18next-browser-languageDetector (GitHub)](https://github.com/i18next/i18next-browser-languageDetector)
- React i18next tutorials: [Crowdin](https://crowdin.com/blog/react-i18n), [Lokalise](https://lokalise.com/blog/how-to-internationalize-react-application-using-i18next/), [Language Unlimited](https://www.languagesunlimited.com/react-i18next-documentation/)
- CLDR plural rules (hi/kn = one/other, `i=0 or n=1`): [Unicode CLDR Language Plural Rules chart v48](https://www.unicode.org/cldr/charts/48/supplemental/language_plural_rules.html), [CLDR plural-rules spec](https://cldr.unicode.org/index/cldr-spec/plural-rules)
- Fonts (install/subsets/license): [Fontsource Noto Sans Kannada](https://fontsource.org/fonts/noto-sans-kannada) + [install](https://fontsource.org/fonts/noto-sans-kannada/install), [npm @fontsource/noto-sans-devanagari](https://www.npmjs.com/package/@fontsource/noto-sans-devanagari), [npm @fontsource/noto-sans-kannada](https://www.npmjs.com/package/@fontsource/noto-sans-kannada), [Noto usage/license docs](https://notofonts.github.io/noto-docs/website/use/)
- OFL-1.1 commercial bundling: [SIL OFL FAQ](https://openfontlicense.org/ofl-faq/)
- Tiny vanilla i18n pattern: [Andreas Remdt ? small i18n script](https://andreasremdt.com/blog/building-a-super-small-and-simple-i18n-script-in-javascript/)
- Low-literacy / India UX: [ICTworks](https://www.ictworks.org/traits-low-literacy-technology-users/), [UX Bulletin](https://www.ux-bulletin.com/designing-low-digital-literacy-users/), [Medhi-Thies et al. (UW PDF)](https://courses.cs.washington.edu/courses/cse490c/18au/readings/medhi-thies-2015.pdf), [Next-billion users (Medium)](https://medium.com/@meinbhidesigner1/crafting-ux-for-indias-next-billion-digital-users-141b07f55210), [Designing for non-tech-savvy users](https://lightweightsolutions.co/building-ui-ux-for-non-tech-savvy-users-designing-for-digital-literacy/), [WhatsApp low-literacy study (ACM SIGACCESS)](https://dl.acm.org/doi/full/10.1145/3663547.3746389)